

HO CHI MINH UNIVERSITY OF TECHNOLOGY

ELECTRICAL ELECTRONIC ENGINEERING

Electronic Department

-----o0o-----



LOGIC DESIGN / LOGIC SYNTHESIS

32-bit Floating Point Unit (FPU)

INSTRUCTORS: PhD. Tran Hoang Linh

Nguyen Tuan Hung

STUDENT: Pham Gia Khanh 2051132

Bui Nguyen Thanh Long 2051140

HO CHI MINH CITY, MAY, 2025

CATALOG

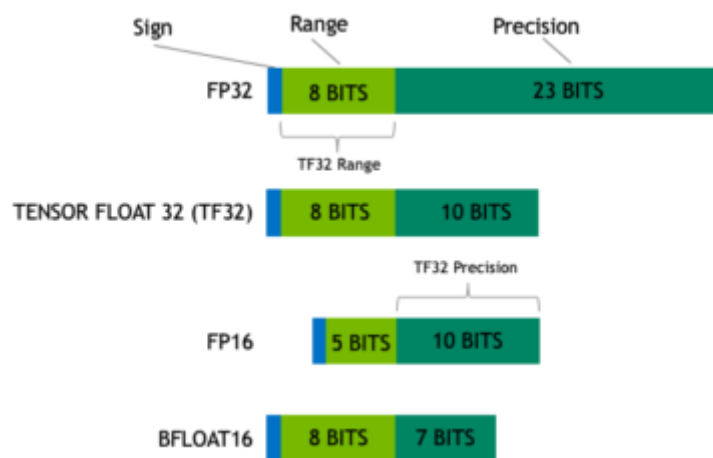
I. Introduction	1
1.1. Nvidia's TensorFlow-32 Format	1
1.2. Overview of sign, exponent, mantissa	1
1.3. Special values	1
1.4. Normalization and denormalization of floating-point values	2
II. DESIGN STRATEGY	3
2.1. Implementation General of Top Module	3
2.2. Implementation Details of Hierarchy Modules	4
III. SYNTHESIS USING QUARTUS	
IV. SIMULATION ON QUESTA INTEL	7
V. CONCLUSION	9
VI. REFERENCES	9

I. Introduction

1.1. Nvidia's TensorFloat-32 format

The TF32 uses the same 10-bit mantissa as the half-precision (FP16) math, shown to have more than sufficient margin for the precision requirements of AI workloads. And TF32 adopts the same 8-bit exponent as FP32, so it can support the same numeric range.

The combination makes TF32 a great alternative to FP32 for crunching through single-precision math, specifically the massive multiply-accumulate functions at the heart of deep learning and many HPC apps.



1.2. Overview of sign, exponent, mantissa

The float is converted to a 19-bit pattern in the TF32 floating-point format. Referring to Figure, a TF32 floating-point number consists of three parts: s, e, and f.

1.3. Special values

The 1-bit s (sign) indicates the sign of the number: $s = 1$ for a negative number and $s = 0$ for a nonnegative number. The 8-bit e (exponent) is the biased exponent with the bias of 127. The 10-bit f (fraction) is the significand (mantissa). The value (V) of a single-precision floating-point number is defined by IEEE as follows:

- Normalized : If $0 < e < 255$, then $V = (-1)^s \times 1. f \times 2^{e-127}$;
- $+0, -0$: If $e = 0$ and $f = 0$, then $V = (-1)^s \times 0$;
- Denormalized : If $e = 0$ and $f \neq 0$, then $V = (-1)^s \times 0. f \times 2^{-126}$;
- $+\infty, -\infty$: If $e = 255$ and $f = 0$, then $V = (-1)^s \infty$;
- NaN (not a number): If $e = 255$ and $f \neq 0$, then $V = \text{NaN}$.

1.4. Normalization and denormalization of floating-point values

For a normalized number, the 1 of 1. f is not stored in the 19-bit data; it is called a hidden bit. For a denormalized number, the hidden bit is a 0. Some examples of the single-precision floating-point numbers are shown below.

- Normalized numbers ($V = (-1)^s \times 1. f \times 2^{e-127}$):
 - 1 01111110 1000000000 = $-1.1 \times 2^{126-127} = -0.75$
 - 0 00000001 0000000000 = $+1.0 \times 2^{1-127} = +2^{-126}$
 - 0 11111110 0000000000 = $+1.0 \times 2^{254-127} = +2^{127}$
 - 0 11111110 1111111111 = $+1.1111111111 \times 2^{254-127}$
- Zero, infinity, and NaN:
 - 0 00000000 0000000000 = $+0$
 - 1 00000000 0000000000 = -0
 - 0 11111111 0000000000 = $+\infty$
 - 1 11111111 0000000000 = $-\infty$
 - 0 11111111 1000000000 = NaN
 - 1 11111111 0000010000 = NaN
- Denormalized numbers ($V = (-1)^s \times 0. f \times 2^{-126}$):
 - 0 00000000 1000000000 = $+0.1 \times 2^{-126} = +2^{-127}$
 - 0 00000000 0000000001 = $+2^{-10} \times 2^{-126} = +2^{-136}$

The following steps show how to convert a real number in decimal format to the TF32 floating-point format.

$$\begin{aligned}
 V &= -6.25_{10} = -(6_{10} + 0.25_{10}) = -(110_2 + 0.01_2) = -110.01_2 = -1.1001_2 \times 2^2 \\
 &= -1.1001_2 \times 2^{129-127} = (-1)^s \times 1. f \times 2^{e-127}
 \end{aligned}$$

II. DESIGN STRATEGY

2.1. Implementation General of Top Module

Interface

- Two 19-bit input ports for operands
- One 19-bit output port for the result

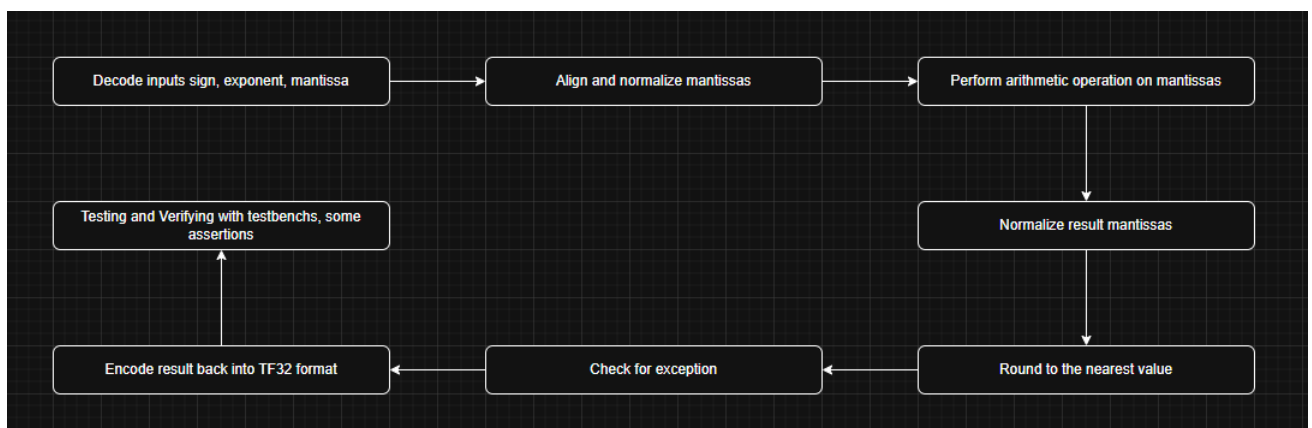
Input signals to specify:

- Operation (+, −, *, /)
- Output signals for exception flags

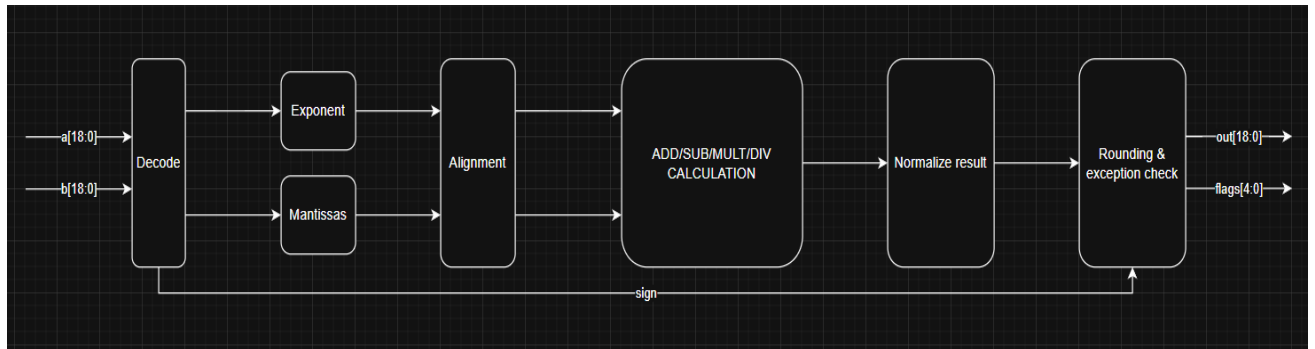
Implementation Guidelines

- Decode inputs into sign, exponent, and mantissa
- Align and normalize mantissas
- Perform arithmetic on mantissas
- Normalize the result mantissa
- Round the mantissa to the nearest value by:
 - + Truncating LSB if 0
 - + Adding 1 and truncating if LSB is 1
- Check for exceptions
- Encode result into Nvidia TensorFloat-32 format
- Testing
- Verifying

Based on the above implementation guidelines, we have the overall flowchart as below. In this lab, we follow this flowchart to construct, verify, and test our FPU. However, this is just in general. For details, we would like to discuss in the following session with each module.

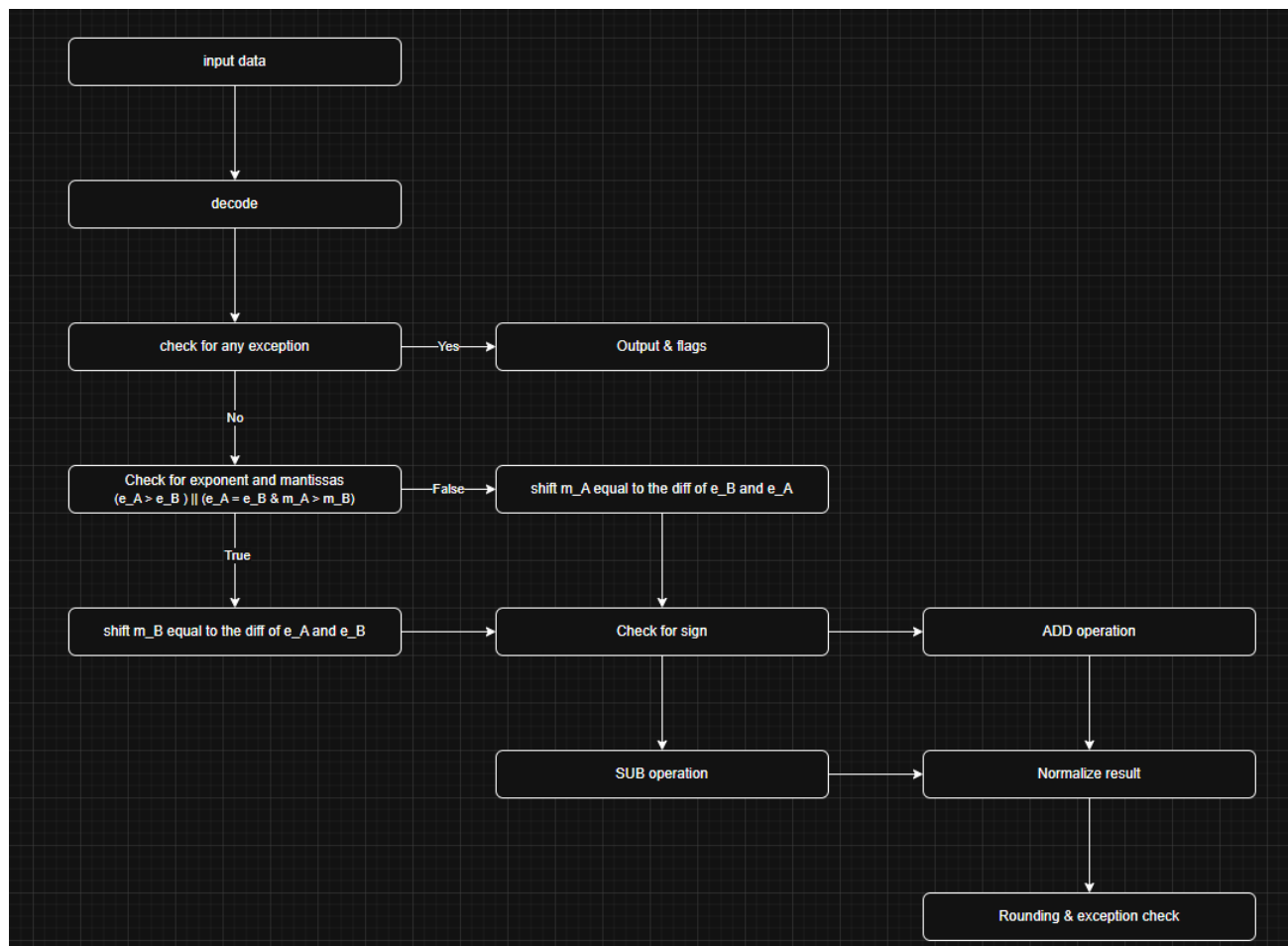


2.2. Implementation Details of Hierarchy Modules



The above figure is a summary of the overall flow chart of the FPU. The number will get decoded into the exponent and mantissa form of the represented numbers. Then it will perform the calculation based on the operation input. After that, the result is normalized into a floating-point format and fed into the Rounding module for rounding and exception checking.

Flow chart of ADD/SUB:

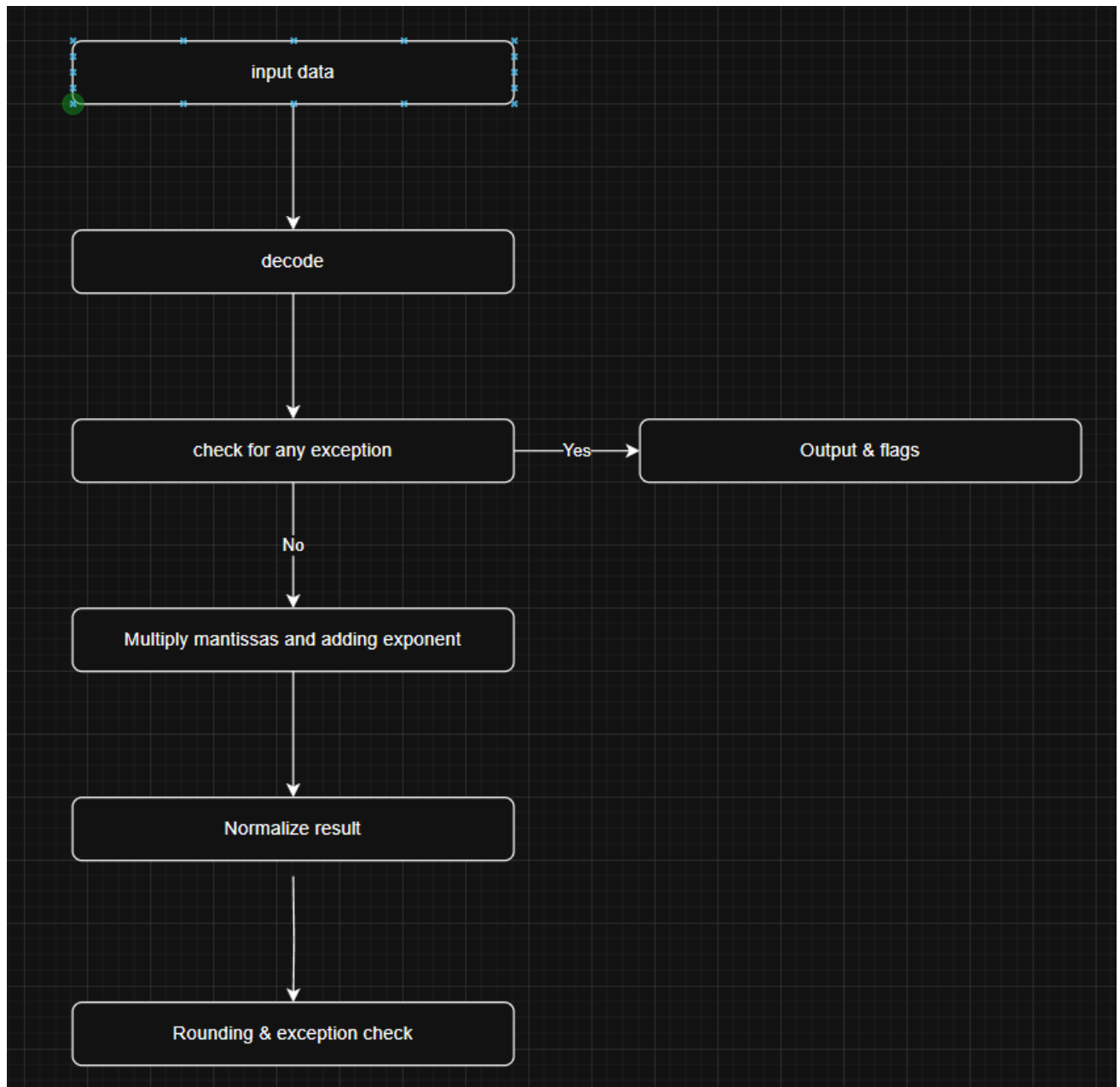


When adding/subtracting two floating-point numbers, there will be 4 cases that can happen:

- Case 1: exponent A > exponent B
- Case 2: exponent A > exponent B
- Case 3: exponent A = exponent B, Precision A > Precision B
- Case 4: exponent A = exponent B, Precision B > Precision A

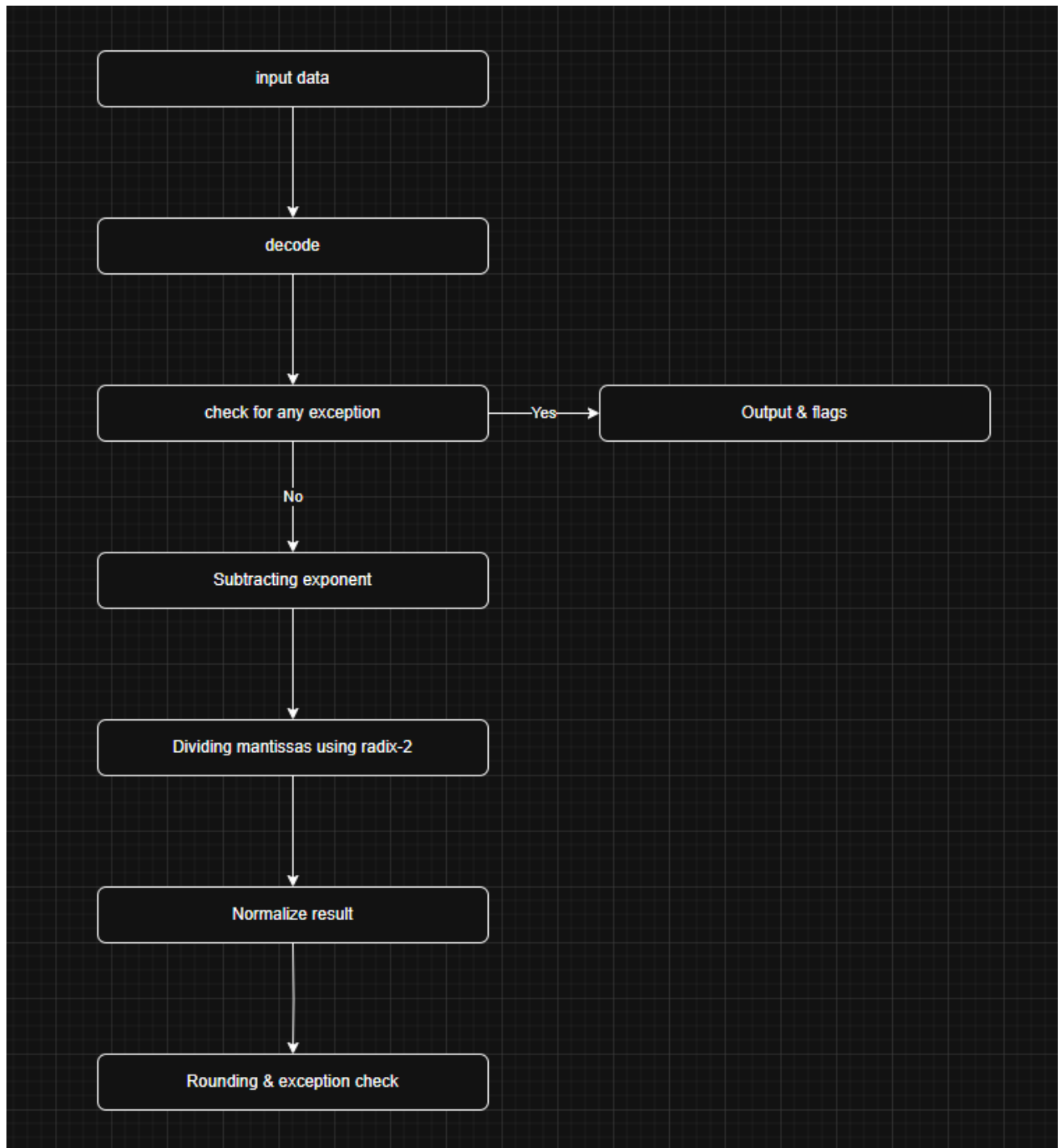
We compute these cases and then shift B/A accordingly to the difference of exponent between the two, and take the higher exponent as the output exponent. The SUB/ADD algorithm will be decided by the sign of A and B. If the two inputs are of the same sign, the module will perform addition, while in contrast, it will perform subtraction. The sign of the component will be decided by the higher value in each case list above. Then the result will be normalized into the right format before into the rounding module and exception check.

Flow chart of MULT



When multiplying two floating-point numbers. First, we will need to add the exponent of the input data and the multiplication operation on the mantissas. Then the result will be normalized for rounding and exception checking. The sign of the output will be determined by the result of $\text{sign_A} \oplus \text{sign_B}$. The XOR is to check if the input sign is the same, or else, the output sign will be positive (0) or (1) for negative.

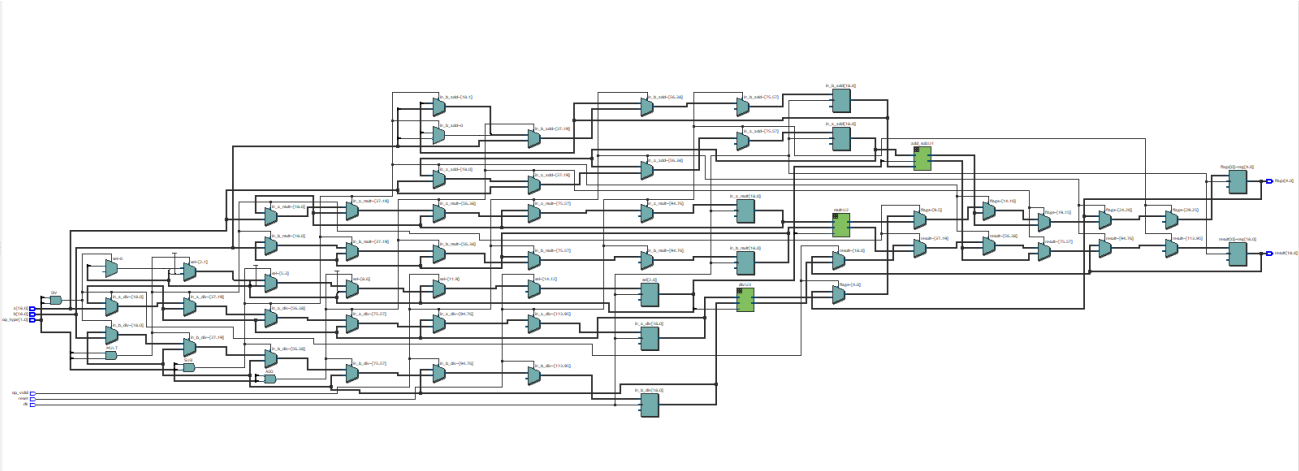
Flowchart of DIV



When dividing two floating-point numbers. Similarly with multiplying the sign will be the value of $\text{sign_A} \text{ xor } \text{sign_B}$. But the output exponent will be the difference between exponent A and exponent B. For dividing the mantissas, we will use the radix-2 algorithm. The algorithm will perform subtraction $m_A - m_B$, and the result will determine the output and the remainder of the division. the loop continues until the last bit of the mantissas. Then the remainder and result will be fed into the rounding module for rounding and exception checking.

III. SYNTHESIS USING QUARTUS

1. RTL nestlist



2. Gate usage

Flow Status	Successful - Fri May 23 22:38:19 2025
Quartus Prime Version	23.1std.1 Build 993 05/14/2024 SC Lite Edition
Revision Name	fpu
Top-level Entity Name	fpu_top
Family	Cyclone V
Device	5CSXFC6D6F31C6
Timing Models	Final
Logic utilization (in ALMs)	551 / 41,910 (1 %)
Total registers	141
Total pins	67 / 499 (13 %)
Total virtual pins	0
Total block memory bits	0 / 5,662,720 (0 %)
Total DSP Blocks	1 / 112 (< 1 %)
Total HSSI RX PCSs	0 / 9 (0 %)
Total HSSI PMA RX Deserializers	0 / 9 (0 %)
Total HSSI TX PCSs	0 / 9 (0 %)
Total HSSI PMA TX Serializers	0 / 9 (0 %)
Total PLLs	0 / 15 (0 %)
Total DLLs	0 / 4 (0 %)

	Resource	Usage
1	Estimate of Logic utilization (ALMs needed)	539
2		
3	▼ Combinational ALUT usage for logic	928
1	-- 7 input functions	15
2	-- 6 input functions	122
3	-- 5 input functions	168
4	-- 4 input functions	284
5	-- <=3 input functions	339
4		
5	Dedicated logic registers	141
6		
7	I/O pins	67
8		
9	Total DSP Blocks	1
10		
11	Maximum fan-out node	reset~input
12	Maximum fan-out	145
13	Total fan-out	4553
14	Average fan-out	3.78

3. Timing

	Fmax	Restricted Fmax	Clock Name	Note
1	36.26 MHz	36.26 MHz	clk	

IV. SIMULATION ON QUESTA-INTEL

For simulation and verification, we will divide each of the operations into three test cases

- Case 1: corner case, the input will be a special value like inf, NaN, and zero.
- Case 2: direct case, the input will be the value that we choose, and operate by hand.
- Case 3: random case, the input will be randomized and computed using Python floating point.

We will check the result using Python programming, and the correct number will be shown, and we will check the output with the correct value by the program. Because the precision is only 10 bits so the number can't be represented most precisely, so we will check for the difference between the correct answer and the output. If the difference by 2 or more, the FPU is wrong; otherwise, it is working fine.

```
E:\py>py fpu_script.py ADD 5
```

```
op = str(sys.argv[1])
numTests = int(sys.argv[2])

with open("fpu_tb.v", 'w') as f:
    if(op == "ADD"):
        f.write("    $display(\"ADD\");\n")
        f.write("    op = 2'b00;\n")
    elif(op == "SUB"):
        f.write("    $display(\"SUB\");\n")
        f.write("    op = 2'b01;\n")
    elif(op == "DIV"):
        f.write("    $display(\"DIV\");\n")
        f.write("    op = 2'b10;\n")
    elif(op == "MULT"):
        f.write("    $display(\"MULT\");\n")
        f.write("    op = 2'b11;\n")

n = 0
while n < numTests:
    try:
        byte = np.random.bytes(4)
        a = np.frombuffer(byte, dtype=np.float32)
        byte = np.random.bytes(4)
        b = np.frombuffer(byte, dtype=np.float32)
        if(op == "ADD"):
            result = a + b
        elif(op == "SUB"):
            result = a - b
        elif(op == "DIV"):
            result = a / b
        elif(op == "MULT"):
            result = a * b

        f.write("    a = 32'b" + binary(a[0]) + ";\n")
        f.write("    b = 32'b" + binary(b[0]) + ";\n")
        f.write("    correct = 32'b" + binary(result[0]) + ";\n")
        f.write("    #400 // " + str(a[0]) + " * " + str(b[0]) + " = " + str(result[0]) + "\n")
        f.write("    $display(\"Output : %b %b %b %h\", out[18], out[17:10], out[9:0], out);\n")
        f.write("    $display(\"Correct: %b %b %b %h\", correct[31], correct[30:23], correct[22:0], correct);\n")
        f.write("    $display();\n")
        n += 1 # only count if success
    except:
        pass # retry
```

There will be 6 assertion cases and 1 normal check to check the FPU

- Case 1: Rst check
- Case 2: invalid check
- Case 3: invalid flag check
- Case 4: divide by zero flag check
- Case 5: overflow flag check
- Case 6: underflow flag check
- Case check 7: correct and output check after we input the data

❖ Floating Point Addition

Corner case

```
$display("ADD");
op = 2'b00;
//corner case
a[31:13] = Nan_def;
b[31:13] = one_def;
correct[31:13] = Nan_def;
#400 //Nan + 1 = Nan
    $display ("Output : %b %b %b %h", out[18], out[17:10], out[9:0], out);
    $display ("Correct: %b %b %b %h",correct[31], correct[30:23], correct[22:0], correct);
    $display();
a[31:13] = zero_def;
b[31:13] = one_def;
correct[31:13] = one_def;
#400 //zero + 1 = 1
    $display ("Output : %b %b %b %h", out[18], out[17:10], out[9:0], out);
    $display ("Correct: %b %b %b %h",correct[31], correct[30:23], correct[22:0], correct);
    $display();
a[31:13] = inf_def;
b[31:13] = inf_def;
correct[31:13] = inf_def;
#400 //inf + inf = inf
    $display ("Output : %b %b %b %h", out[18], out[17:10], out[9:0], out);
    $display ("Correct: %b %b %b %h",correct[31], correct[30:23], correct[22:0], correct);
    $display();
a[31:13] = inf_def;
b[31:13] = one_def;
correct[31:13] = inf_def;
#400 //inf + 1 = inf
    $display ("Output : %b %b %b %h", out[18], out[17:10], out[9:0], out);
    $display ("Correct: %b %b %b %h",correct[31], correct[30:23], correct[22:0], correct);
    $display();
....
```

Direct case

```
//direct case
$display("ADD direct case");
a[31:13] = 19'b0100001011110000000;
b[31:13] = 19'b0100010000010110000;
correct[31:13] = 19'b0100010000110100000;
#400 //120 + 600 = 720
    $display ("Output : %b %b %b %h", out[18], out[17:10], out[9:0], out);
    $display ("Correct: %b %b %b %h",correct[31], correct[30:23], correct[22:0], correct);
    $display();
a[31:13] = 19'b0100001100011110010;
b[31:13] = 19'b0100000010110001100;
correct[31:13] = 19'b0100001100100011110;
#400 //158.25 + 5.55 = 163.8
    $display ("Output : %b %b %b %h", out[18], out[17:10], out[9:0], out);
    $display ("Correct: %b %b %b %h",correct[31], correct[30:23], correct[22:0], correct);
    $display();
a[31:13] = 19'b0100001011111010101;
b[31:13] = 19'b1100001011100011111;
correct[31:13] = 19'b0100000100110110001;
#400 //125.34 + -113.95 = 11.39
    $display ("Output : %b %b %b %h", out[18], out[17:10], out[9:0], out);
    $display ("Correct: %b %b %b %h",correct[31], correct[30:23], correct[22:0], correct);
    $display();
....
```

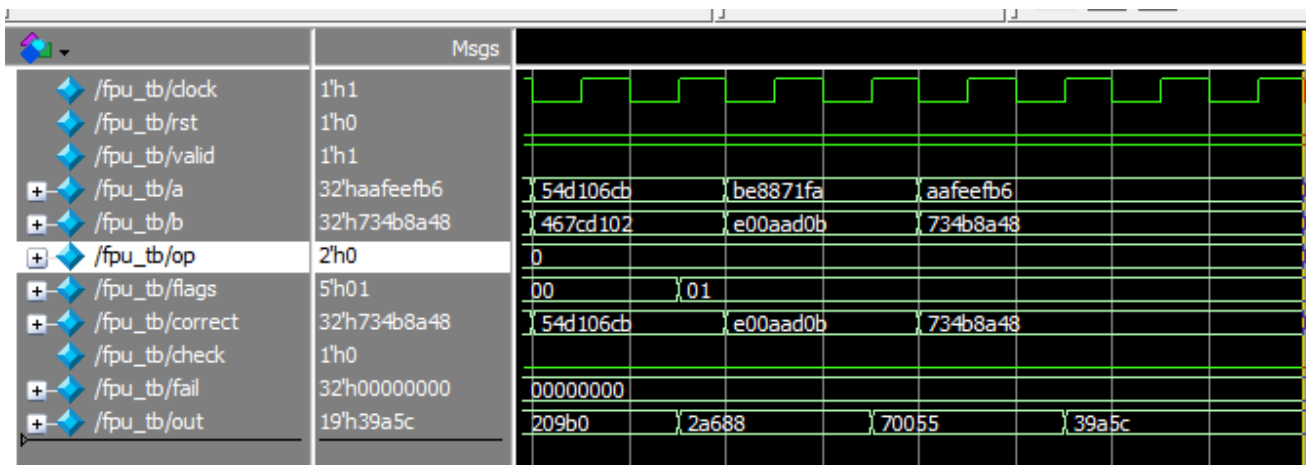
Random case

```
//random case
$display("ADD random case");
a = 32'b01010100110100010000011011001011;
b = 32'b01000110011111001101000100000010;
correct = 32'b01010100110100010000011011001011;
#400 //7182097000000.0 * 16180.252 = 7182097000000.0
$display ("Output : %b %b %b %h", out[18], out[17:10], out[9:0], out);
$display ("Correct: %b %b %b %h",correct[31], correct[30:23], correct[22:0], correct);
$display();
a = 32'b1011111010001000011100011111010;
b = 32'b1110000000001010101010100001011;
correct = 32'b1110000000001010101010100001011;
#400 //-0.26649457 * -3.997062e+19 = -3.997062e+19
$display ("output : %b %b %b %h", out[18], out[17:10], out[9:0], out);
$display ("Correct: %b %b %b %h",correct[31], correct[30:23], correct[22:0], correct);
$display();
a = 32'b101010101111110111011110110110;
b = 32'b01110011010010111000101001001000;
correct = 32'b01110011010010111000101001001000;
#400 //-4.5285797e-13 * 1.6126113e+31 = 1.6126113e+31
$display ("Output : %b %b %b %h", out[18], out[17:10], out[9:0], out);
$display ("Correct: %b %b %b %h",correct[31], correct[30:23], correct[22:0], correct);
$display();
-----
```

Simulation result

```
# ADD
# Output : 0 11111111 1000000000 3fe00
# Correct: 0 11111111 1000000000000000000000 7fc00000
#
# Output : 0 01111111 0000000000 1fc00
# Correct: 0 01111111 0000000000000000000000 3f800000
#
# Output : 0 11111111 0000000000 3fc00
# Correct: 0 11111111 0000000000000000000000 7f800000
#
# Output : 0 11111111 0000000000 3fc00
# Correct: 0 11111111 0000000000000000000000 7f800000
#
# ADD direct case
# Output : 0 10001000 0110100000 221a0
# Correct: 0 10001000 0110100000000000000000 44340000
#
# Output : 0 10000110 0100011110 2191e
# Correct: 0 10000110 0100011110000000000000 4323c000
#
# Output : 0 10000010 0110110000 209b0
# Correct: 0 10000010 0110110001000000000000 41362000
#
# ADD random case
# Output : 0 10101001 1010001000 2a688
# Correct: 0 10101001 10100010000011011001011 54d106cb
#
# Output : 1 11000000 0001010101 70055
# Correct: 1 11000000 00010101010110100001011 e00aad0b
#
# Output : 0 11100110 1001011100 39a5c
# Correct: 0 11100110 10010111000101001001000 734b8a48
#
```

Waveform of random cases



❖ Floating point Subtraction

Corner case

```
$display("SUB");
op = 2'b01;
correct[12:0] = 0;
//corner case
a[31:13] = Nan_def;
b[31:13] = one_def;
correct[31:13] = Nan_def;
#400 //Nan - 1 = Nan
    $display ("Output : %b %b %b %h", out[18], out[17:10], out[9:0], out);
    $display ("Correct: %b %b %b %h",correct[31], correct[30:23], correct[22:0], correct);
    $display();
a[31:13] = one_def;
b[31:13] = zero_def;
correct[31:13] = one_def;
#400 //1 - zero = 1
    $display ("Output : %b %b %b %h", out[18], out[17:10], out[9:0], out);
    $display ("Correct: %b %b %b %h",correct[31], correct[30:23], correct[22:0], correct);
    $display();
a[31:13] = inf_def;
b[31:13] = inf_def;
correct[31:13] = Nan_def;
#400 //inf - inf = Nan
    $display ("Output : %b %b %b %h", out[18], out[17:10], out[9:0], out);
    $display ("Correct: %b %b %b %h",correct[31], correct[30:23], correct[22:0], correct);
    $display();
a[31:13] = inf_def;
b[31:13] = one_def;
correct[31:13] = inf_def;
#400 //inf - 1 = inf
    $display ("Output : %b %b %b %h", out[18], out[17:10], out[9:0], out);
    $display ("Correct: %b %b %b %h",correct[31], correct[30:23], correct[22:0], correct);
    $display();
//end
```

Direct case

```
//direct case
$display("SUB direct case");
a[31:13] = 19'b0100010001001000000;
b[31:13] = 19'b0100001101110000000;
correct[31:13] = 19'b0100010000001100000;
#400 //800 - 240 = 560
    $display ("Output : %b %b %b %h", out[18], out[17:10], out[9:0], out);
    $display ("Correct: %b %b %b %h",correct[31], correct[30:23], correct[22:0], correct);
    $display();
a[31:13] = 19'b01000010111001001000;
b[31:13] = 19'b0011111100001100110;
correct[31:13] = 19'b0100001011000111111;
#400 //100.5 - 0.55 = 99.95
    $display ("Output : %b %b %b %h", out[18], out[17:10], out[9:0], out);
    $display ("Correct: %b %b %b %h",correct[31], correct[30:23], correct[22:0], correct);
    $display();
a[31:13] = 19'b1100001011111010101;
b[31:13] = 19'b0100001011100011111;
correct[31:13] = 19'b1100001101101111010;
//end
```

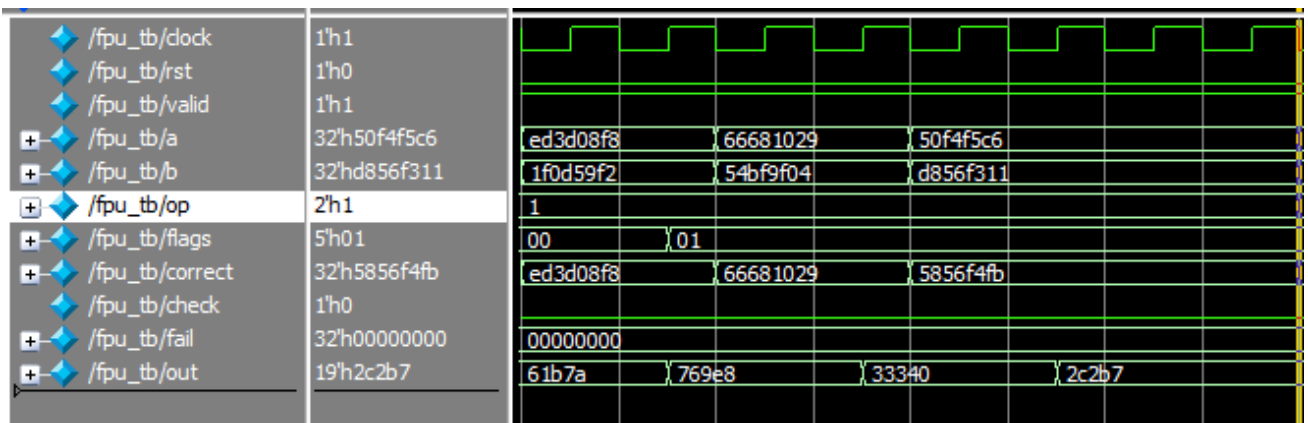
Random case

```
//random case
$display("SUB random case");
a = 32'b111011010011111010000100011111000;
b = 32'b00011111000011010101100111110010;
correct = 32'b111011010011111010000100011111000;
#400 //-3.6564693e+27 * 2.9932312e-20 = -3.6564693e+27
    $display ("Output : %b %b %b %h", out[18], out[17:10], out[9:0], out);
    $display ("Correct: %b %b %b %h",correct[31], correct[30:23], correct[22:0], correct);
    $display();
a = 32'b01100110011010000001000000101001;
b = 32'b0101010010111111001111100000100;
correct = 32'b01100110011010000001000000101001;
#400 //2.7397178e+23 * 6584053000000.0 = 2.7397178e+23
    $display ("Output : %b %b %b %h", out[18], out[17:10], out[9:0], out);
    $display ("Correct: %b %b %b %h",correct[31], correct[30:23], correct[22:0], correct);
    $display();
a = 32'b01010000111101001111010111000110;
b = 32'b11011000010101101111001100010001;
correct = 32'b0101100001010110111101001111011;
#400 //32877982000.0 * -945357800000000.0 = 945390700000000.0
    $display ("Output : %b %b %b %h", out[18], out[17:10], out[9:0], out);
    $display ("Correct: %b %b %b %h",correct[31], correct[30:23], correct[22:0], correct);
    $display();
#400
```

Simulation result

```
# SUB
# Output : 0 11111111 1000000000 3fe00
# Correct: 0 11111111 1000000000000000000000 7fc00000
#
# Output : 0 01111111 0000000000 1fc00
# Correct: 0 01111111 0000000000000000000000 3f800000
#
# Output : 0 11111111 0000000000 3fc00
# Correct: 0 11111111 1000000000000000000000 7fc00000
#
# Output : 0 11111111 0000000000 3fc00
# Correct: 0 11111111 0000000000000000000000 7f800000
#
# SUB direct case
# Output : 0 10001000 0001100000 22060
# Correct: 0 10001000 0001100000000000000000 440c0000
#
# Output : 0 10000101 1001000000 21640
# Correct: 0 10000101 100011111100000000000000 42c7e000
#
# Output : 1 10000110 1101111010 61b7a
# Correct: 1 10000110 110111101000000000000000 c36f4000
#
# SUB random case
# Output : 1 11011010 0111101000 769e8
# Correct: 1 11011010 01111010000100011111000 ed3d08f8
#
# Output : 0 11001100 1101000000 33340
# Correct: 0 11001100 11010000001000000101001 66681029
#
# Output : 0 10110000 1010110111 2c2b7
# Correct: 0 10110000 10101101111010011111011 5856f4fb
#
```

Waveform of random cases



❖ Floating Point Multiplication

Corner case

```
$display("MULT");
op = 2'b11;
correct[12:0] = 0;
//corner case
a[31:13] = Nan_def;
b[31:13] = one_def;
correct[31:13] = Nan_def;
#400 //Nan * 1 = Nan
    $display ("Output : %b %b %b %h", out[18], out[17:10], out[9:0], out);
    $display ("Correct: %b %b %b %h",correct[31], correct[30:23], correct[22:0], correct);
    $display();
a[31:13] = zero_def;
b[31:13] = one_def;
correct[31:13] = zero_def;
#400 //zero * 1 = zero
    $display ("Output : %b %b %b %h", out[18], out[17:10], out[9:0], out);
    $display ("Correct: %b %b %b %h",correct[31], correct[30:23], correct[22:0], correct);
    $display();
a[31:13] = inf_def;
b[31:13] = inf_def;
correct[31:13] = inf_def;
#400 //inf * inf = inf
    $display ("Output : %b %b %b %h", out[18], out[17:10], out[9:0], out);
    $display ("Correct: %b %b %b %h",correct[31], correct[30:23], correct[22:0], correct);
    $display();
a[31:13] = inf_def;
b[31:13] = one_def;
correct[31:13] = inf_def;
#400 //inf * 1 = inf
    $display ("Output : %b %b %b %h", out[18], out[17:10], out[9:0], out);
    $display ("Correct: %b %b %b %h",correct[31], correct[30:23], correct[22:0], correct);
    $display();
a[31:13] = inf_def;
b[31:13] = zero_def;
correct[31:13] = Nan_def;
#400 //inf * 1 = inf
    $display ("Output : %b %b %b %h", out[18], out[17:10], out[9:0], out);
    $display ("Correct: %b %b %b %h",correct[31], correct[30:23], correct[22:0], correct);
    $display();
....
```

Direct case

```
....
//direct case
$display("MULT direct case");
a[31:13] = 19'b0100111010001111000;
b[31:13] = 19'b0101000101011111100;
correct[31:13] = 19'b0110000001111001110;
#400 //12e8 * 6e10 = 72 e18
    $display ("Output : %b %b %b %h", out[18], out[17:10], out[9:0], out);
    $display ("Correct: %b %b %b %h",correct[31], correct[30:23], correct[22:0], correct);
    $display();
a[31:13] = 19'b0100001011100110110;
b[31:13] = 19'b0100000100110100000;
correct[31:13] = 19'b0100010010100010001;
#400 //115.375 * 11.25 = 1297.96875
    $display ("Output : %b %b %b %h", out[18], out[17:10], out[9:0], out);
    $display ("Correct: %b %b %b %h",correct[31], correct[30:23], correct[22:0], correct);
    $display();
a[31:13] = 19'b0100011111110100110;
b[31:13] = 19'b0011110111100101011;
correct[31:13] = 19'b0100011001011011010;
#400 //125.3422e3 * 0.112 = 14038.3264
    $display ("Output : %b %b %b %h", out[18], out[17:10], out[9:0], out);
    $display ("Correct: %b %b %b %h",correct[31], correct[30:23], correct[22:0], correct);
    $display();
..
```

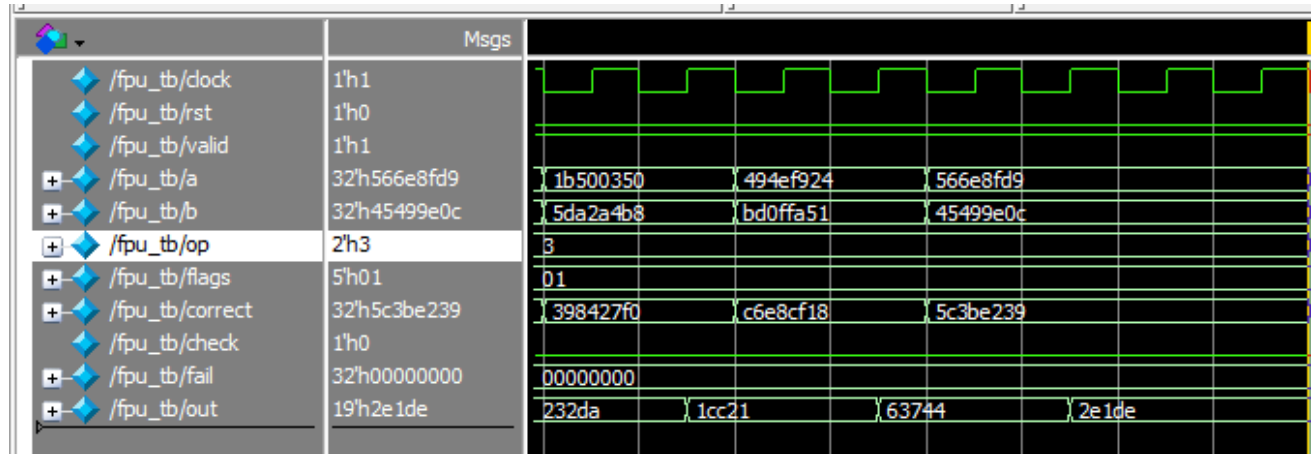

Random case

```
//random case
$display("MULT random case");
a = 32'b0001101101010000000001101010000;
b = 32'b010111011010001010010010111000;
correct = 32'b0011100110000100001001111110000;
#400 //1.7206427e-22 * 1.4649618e+18 = 0.00025206758
$display ("Output : %b %b %b %h", out[18], out[17:10], out[9:0], out);
$display ("Correct: %b %b %b %h",correct[31], correct[30:23], correct[22:0], correct);
$display();
a = 32'b0100100101001110111100100100100;
b = 32'b1011110100001111111101001010001;
correct = 32'b11000110111010001100111100011000;
#400 //847762.25 * -0.03515083 = -29799.547
$display ("Output : %b %b %b %h", out[18], out[17:10], out[9:0], out);
$display ("Correct: %b %b %b %h",correct[31], correct[30:23], correct[22:0], correct);
$display();
a = 32'b01010110011011101000111111011001;
b = 32'b01000101010010011001111000001100;
correct = 32'b01011100001110111110001000111001;
#400 //65575397000000.0 * 3225.878 = 2.1153822e+17
$display ("Output : %b %b %b %h", out[18], out[17:10], out[9:0], out);
$display ("Correct: %b %b %b %h",correct[31], correct[30:23], correct[22:0], correct);
$display();
```

Simulation result

```
|
| MULT
| Output : 0 11111111 1000000000 3fe00
| Correct: 0 11111111 1000000000000000000000 7fc00000
|
| Output : 0 00000000 0000000000 00000
| Correct: 0 00000000 0000000000000000000000 00000000
|
| Output : 0 11111111 0000000000 3fc00
| Correct: 0 11111111 0000000000000000000000 7f800000
|
| Output : 0 11111111 0000000000 3fc00
| Correct: 0 11111111 0000000000000000000000 7f800000
|
| Output : 0 11111111 1000000000 3fe00
| Correct: 0 11111111 1000000000000000000000 7fc00000
|
| MULT direct case
| Output : 0 11000000 1111001110 303ce
| Correct: 0 11000000 111100111000000000000000 6079c000
|
| Output : 0 10001001 0100010010 22512
| Correct: 0 10001001 010001000100000000000000 44a22000
|
| Output : 0 10001100 1011011010 232da
| Correct: 0 10001100 101101101000000000000000 465b4000
|
| MULT random case
| Output : 0 01110011 0000100001 1cc21
| Correct: 0 01110011 00001000010011111110000 398427f0
|
| Output : 1 10001101 1101000100 63744
| Correct: 1 10001101 110100011001111100011000 c6e8cf18
|
| Output : 0 10111000 0111011110 2elde
| Correct: 0 10111000 01110111110001000111001 5c3be239
|
```

Waveform of random cases



❖ Floating Point Division

Corner case

```
//DIV
$display("DIV");
op = 2'b10;
correct[12:0] = 0;
//corner case
a[31:13] = Nan_def;
b[31:13] = one_def;
correct[31:13] = Nan_def;
#400 //Nan / 1 = Nan
$display ("Output : %b %b %b %h", out[18], out[17:10], out[9:0], out);
$display ("Correct: %b %b %b %h", correct[31], correct[30:23], correct[22:0], correct);
$display();
a[31:13] = zero_def;
b[31:13] = one_def;
correct[31:13] = zero_def;
#400 //zero / 1 = zero
$display ("Output : %b %b %b %h", out[18], out[17:10], out[9:0], out);
$display ("Correct: %b %b %b %h", correct[31], correct[30:23], correct[22:0], correct);
$display();
a[31:13] = one_def;
b[31:13] = zero_def;
correct[31:13] = inf_def;
#400 //1 / zero = inf
$display ("Output : %b %b %b %h", out[18], out[17:10], out[9:0], out);
$display ("Correct: %b %b %b %h", correct[31], correct[30:23], correct[22:0], correct);
$display();
a[31:13] = zero_def;
b[31:13] = zero_def;
correct[31:13] = Nan_def;
#400 //zero / zero = Nan
$display ("Output : %b %b %b %h", out[18], out[17:10], out[9:0], out);
$display ("Correct: %b %b %b %h", correct[31], correct[30:23], correct[22:0], correct);
$display();
a[31:13] = inf_def;
b[31:13] = inf_def;
correct[31:13] = Nan_def;
#400 //inf / inf = NaN
$display ("Output : %b %b %b %h", out[18], out[17:10], out[9:0], out);
$display ("Correct: %b %b %b %h", correct[31], correct[30:23], correct[22:0], correct);
$display();
a[31:13] = inf_def;
b[31:13] = one_def;
correct[31:13] = inf_def;
#400 //inf / 1 = inf
$display ("Output : %b %b %b %h", out[18], out[17:10], out[9:0], out);
$display ("Correct: %b %b %b %h", correct[31], correct[30:23], correct[22:0], correct);
$display();
a[31:13] = one_def;
b[31:13] = inf_def;
correct[31:13] = zero_def;
#400 //1 / inf = zero
$display ("Output : %b %b %b %h", out[18], out[17:10], out[9:0], out);
$display ("Correct: %b %b %b %h", correct[31], correct[30:23], correct[22:0], correct);
$display();
....
```

Direct case

```

//direct case
$display("DIV direct case");
a[31:13] = 19'b010100011101111100;
b[31:13] = 19'b0100111000001111000;
correct[31:13] = 19'b0100001101001000000;
#400 //12 e10 / 6 e8 = 2 e2
$display ("Output : %b %b %b %h", out[18], out[17:10], out[9:0], out);
$display ("Correct: %b %b %b %h",correct[31], correct[30:23], correct[22:0], correct);
$display();
a[31:13] = 19'b0100001100101111010;
b[31:13] = 19'b0100000010101000000;
correct[31:13] = 19'b0100001000000101100;
#400 //175.25 / 5.25 = 33.380952381
$display ("Output : %b %b %b %h", out[18], out[17:10], out[9:0], out);
$display ("Correct: %b %b %b %h",correct[31], correct[30:23], correct[22:0], correct);
$display();
a[31:13] = 19'b0100001011110110101;
b[31:13] = 19'b1100001011001111111;
correct[31:13] = 19'b1011111110010111111;
#400 //123.34 / -103.95 = -1.18653198653
$display ("Output : %b %b %b %h", out[18], out[17:10], out[9:0], out);
$display ("Correct: %b %b %b %h",correct[31], correct[30:23], correct[22:0], correct);
$display();

```

Random case

```

//random case
$display("DIV random case");
a = 32'b000011011111001110000011100101100;
b = 32'b00111011100000011101001111110011;
correct = 32'b00010001111000111100011001110110;
#400 //1.4238201e-30 * 0.003962034 = 3.5936596e-28
$display ("Output : %b %b %b %h", out[18], out[17:10], out[9:0], out);
$display ("Correct: %b %b %b %h",correct[31], correct[30:23], correct[22:0], correct);
$display();
a = 32'b01001110010111110100100101011111;
b = 32'b11111100011110111111011111010001;
correct = 32'b10010001011000101101110000010000;
#400 //936531900.0 * -5.2331713e+36 = -1.7896068e-28
$display ("Output : %b %b %b %h", out[18], out[17:10], out[9:0], out);
$display ("Correct: %b %b %b %h",correct[31], correct[30:23], correct[22:0], correct);
$display();
a = 32'b00100011101110001101010001010010;
b = 32'b00111110110101100101010111100110;
correct = 32'b00100100010111001100001000010111;
#400 //2.0039241e-17 * 0.4186241 = 4.7869296e-17
$display ("Output : %b %b %b %h", out[18], out[17:10], out[9:0], out);
$display ("Correct: %b %b %b %h",correct[31], correct[30:23], correct[22:0], correct);
$display();
$display ("Done.");
$display ("Failed = %0d",fail);
$finish;

```

Simulation result

```
# DIV
# Output : 0 11111111 1000000000 3fe00
# Correct: 0 11111111 1000000000000000000000 7fc00000
#
# Output : 0 00000000 0000000000 00000
# Correct: 0 00000000 0000000000000000000000 00000000
#
# Output : 0 11111111 0000000000 3fc00
# Correct: 0 11111111 0000000000000000000000 7f800000
#
# Output : 0 11111111 1000000000 3fe00
# Correct: 0 11111111 1000000000000000000000 7fc00000
#
# Output : 0 11111111 1000000000 3fe00
# Correct: 0 11111111 1000000000000000000000 7fc00000
#
# Output : 0 11111111 0000000000 3fc00
# Correct: 0 11111111 0000000000000000000000 7f800000
#
# Output : 0 00000000 0000000000 00000
# Correct: 0 00000000 0000000000000000000000 00000000
#
# DIV direct case
# Output : 0 10000110 1001000000 21a40
# Correct: 0 10000110 1001000000000000000000 43480000
#
# Output : 0 10000100 0000101100 2102c
# Correct: 0 10000100 0000101100000000000000 42058000
#
# Output : 1 01111111 0010111111 5fcbf
# Correct: 1 01111111 0010111111000000000000 bf97e000
#
# DIV random case
# Output : 0 00100011 1100011111 08f1f
# Correct: 0 00100011 11000111100011001110110 11e3c676
#
# Output : 1 00100010 1100010111 48b17
# Correct: 1 00100010 11000101101110000010000 9162dc10
#
# Output : 0 01001000 1011100110 122e6
# Correct: 0 01001000 10111001100001000010111 245cc217
```

Waveform of random cases

0de7072c				4e5f495f				23b8d452			
3b81d3f3				fc7bf7d1				3ed655e6			
11e3c676				9162dc10				245cc217			
			08f1f				48b17				122e6

V. CONCLUSION

Design and implement a 19-bit floating point unit (FPU) compliant with Nvidia TensorFloat-32 formatted. It performs basic arithmetic operations on 2-bit floating-point operands, rounding results to the nearest value. We study TF32 format, Overview of sign, exponent, mantissa, special values (infinity, NaN, denormals), normalization and denormalization of floating-point values, handling underflow and overflow conditions. However, we still have not covered all cases that can happen. We will try to improve and develop our algorithm and architecture for future projects, such as an embedded FPU in our RISC-V CPU for floating point instructions or AI accelerator-related projects.

VI. REFERENCES

1. Book: Computer Principles and Design in Verilog HDL
2. Web: <https://blogs.nvidia.com/blog/tensorfloat-32-precision-format/>